

AFRILANG Stdlib

std/c/graphkruskal

Bibliothèque AFRILANG `std/c/graphkruskal` (niveau complexe, catégorie complexe). Elle expose 7 fonction(s)
: kruskalMst,

```
`import "std/c/graphkruskal" . `use graphkruskal`
```

- `kruskalMst(adj list number, n number) ? number`
- `kruskalEdges(adj list number, n number) ? list number`
- `unionFindComponents(adj list number, n number) ? number`
- `edgeListSize(adj list number, n number) ? number`
- `kruskalForestWeight(adj list number, n number) ? number`
- `maxSpanningForest(adj list number, n number) ? number`
- `compareKruskalPrim(adj list number, n number) ? number`

Category: complexe | Tier: complexe | Fonctions: 7

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/graphkruskal` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : `kruskalMst`,

```
`import "std/c/graphkruskal" . `use graphkruskal`  
- `kruskalMst(adj list number, n number) ? number`  
- `kruskalEdges(adj list number, n number) ? list number`  
- `unionFindComponents(adj list number, n number) ? number`  
- `edgeListSize(adj list number, n number) ? number`  
- `kruskalForestWeight(adj list number, n number) ? number`  
- `maxSpanningForest(adj list number, n number) ? number`  
- `compareKruskalPrim(adj list number, n number) ? number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/graphkruskal"  
use graphkruskal
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? kruskalMst(adj list number, n number) ? number  
? kruskalEdges(adj list number, n number) ? list  
? unionFindComponents(adj list number, n number) ? number  
? edgeListSize(adj list number, n number) ? number  
? kruskalForestWeight(adj list number, n number) ? number  
? maxSpanningForest(adj list number, n number) ? number  
? compareKruskalPrim(adj list number, n number) ? number
```

4. Exemples d'utilisation

```
import "std/c/graphkruskal"  
use graphkruskal  
  
create result = kruskalMst(/* args */)   
say result  
  
create result = kruskalEdges(/* args */)   
say result  
  
create result = unionFindComponents(/* args */)   
say result  
  
create result = edgeListSize(/* args */)   
say result  
  
create result = kruskalForestWeight(/* args */)   
say result  
  
create result = maxSpanningForest(/* args */)   
say result
```

5. Bonnes pratiques

```
? Préférez `import "std/c/graphkruskal"` en tête de fichier.  
? Lancez `afrilang check` avant de livrer.  
? Combinez avec les modules stdlib de la même catégorie.  
? Voir /stdlib/ pour le source live et les démos playground.  
? Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe ? source

```
module graphkruskal
```

```
function kruskalMst(adj list number, n number) returns number
end

function kruskalEdges(adj list number, n number) returns list number
end

function unionFindComponents(adj list number, n number) returns number
end

function edgeListSize(adj list number, n number) returns number
end

function kruskalForestWeight(adj list number, n number) returns number
end

function maxSpanningForest(adj list number, n number) returns number
end

function compareKruskalPrim(adj list number, n number) returns number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/graphkruskal` (tier complex, category complex). Exports 7 function(s):
kruskalMst, kruskalEdges,

```
`import "std/c/graphkruskal" . `use graphkruskal`  
- `kruskalMst(adj list number, n number) ? number`  
- `kruskalEdges(adj list number, n number) ? list number`  
- `unionFindComponents(adj list number, n number) ? number`  
- `edgeListSize(adj list number, n number) ? number`  
- `kruskalForestWeight(adj list number, n number) ? number`  
- `maxSpanningForest(adj list number, n number) ? number`  
- `compareKruskalPrim(adj list number, n number) ? number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/graphkruskal"  
use graphkruskal
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? kruskalMst(adj list number, n number) ? number  
? kruskalEdges(adj list number, n number) ? list  
? unionFindComponents(adj list number, n number) ? number  
? edgeListSize(adj list number, n number) ? number  
? kruskalForestWeight(adj list number, n number) ? number  
? maxSpanningForest(adj list number, n number) ? number  
? compareKruskalPrim(adj list number, n number) ? number
```

4. Usage examples

```
import "std/c/graphkruskal"  
use graphkruskal  
  
create result = kruskalMst(/* args */)   
say result  
  
create result = kruskalEdges(/* args */)   
say result  
  
create result = unionFindComponents(/* args */)   
say result  
  
create result = edgeListSize(/* args */)   
say result  
  
create result = kruskalForestWeight(/* args */)   
say result  
  
create result = maxSpanningForest(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/graphkruskal" .` at the top of your file.  
? Run `afriLang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module graphkruskal
```

```
function kruskalMst(adj list number, n number) returns number
end

function kruskalEdges(adj list number, n number) returns list number
end

function unionFindComponents(adj list number, n number) returns number
end

function edgeListSize(adj list number, n number) returns number
end

function kruskalForestWeight(adj list number, n number) returns number
end

function maxSpanningForest(adj list number, n number) returns number
end

function compareKruskalPrim(adj list number, n number) returns number
end
end
```